

Muhamad Lingga Darmawan

152024042

IFB-206 KOMPUTASI PARALEL & SISTEM TERDISTRIBUSI BB

dynamic distribution (for uneven NRP), shows when the code expected optimal time

CODE:

```
def static_uneven_distribution(tasks, num_workers):
    """
    Menyimulasikan distribusi tugas secara statis (Round-Robin) untuk tugas yang
    tidak merata (uneven).
    Menampilkan momen kapan seorang pekerja mencapai/melewati "Ideal
    Distribution".
    """
    print(f"=== STATIC UNEVEN DISTRIBUTION ===")
    print(f"Total Tugas: {len(tasks)} | Total Pekerja: {num_workers}\n")

    # Menghitung Ideal Distribution
    # Idealnya, jika tugas bisa dibagi rata sempurna, setiap pekerja akan mendapat
    # beban ini.
    total_workload = sum(tasks)
    ideal_distribution = total_workload / num_workers

    print(f"Total Seluruh Beban Kerja: {total_workload} detik")
    print(f"[*] IDEAL DISTRIBUTION (Target Rata-rata): {ideal_distribution:.2f}
    detik per pekerja\n")

    workers_load = [0] * num_workers

    # Eksekusi secara statis (dibagi bergantian / Round-Robin)
    for index, task_time in enumerate(tasks):
        # 1. Menentukan pekerja secara statis berdasarkan urutan (index), bukan
        # berdasarkan beban
        assigned_worker = index % num_workers

        # 2. Mendistribusikan tugas ke pekerja tersebut
        workers_load[assigned_worker] += task_time

    # Output Penjelasan Eksekusi
    print(f"[Eksekusi] Tugas ke-{index + 1} (beban {task_time}s) -> Dialokasikan
    secara statis ke Pekerja {assigned_worker + 1}")
    print(f"          Beban Pekerja {assigned_worker + 1} sekarang:
    {workers_load[assigned_worker]}s")
```

3. Menunjukkan KAPAN kode mencapai/melewati Ideal Distribution (Sesuai permintaan soal)

```
if workers_load[assigned_worker] >= ideal_distribution:  
    print(f'>>> [INFO] Pekerja {assigned_worker + 1} telah  
mencapai/melewati batas Ideal Distribution! <<<')
```

```
print("\n=== Hasil Akhir Distribusi (Uneven / Tidak Merata) ===")  
for i, load in enumerate(workers_load):  
    # Menampilkan hasil akhir yang kemungkinan besar tidak merata karena  
distribusinya statis  
    print(f'Pekerja {i + 1} total beban: {load}s (Selisih dari ideal: {abs(load -  
ideal_distribution):.2f}s)')
```

```
print("-" * 55)  
print(f'[*] IDEAL DISTRIBUTION POINT: {ideal_distribution:.2f} detik')  
print("-" * 55)
```

```
if __name__ == "__main__":
```

```
    simulated_tasks = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] # Contoh beban tugas yang tidak  
merata (uneven)
```

```
    print(f'Daftar Waktu Tugas (Uneven): {simulated_tasks}\n')
```

```
    # Menjalankan fungsi dengan 3 pekerja  
    static_uneven_distribution(simulated_tasks, num_workers=3)
```

OUTPUT:

```

PS C:\Users\mling\Documents\Kuliah\kompar\load balancing> & C:/Users/mling/AppData/Local/Programs/Python/Python310/python.exe "c:/Users/mling/Documents/Kuliah/kompar/load balancing/Quiz-Practice-Load balancing.py"
Daftar Waktu Tugas (Uneven): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

=== STATIC UNEVEN DISTRIBUTION ===
Total Tugas: 10 | Total Pekerja: 3

Total Seluruh Beban Kerja: 55 detik
[*] IDEAL DISTRIBUTION (Target Rata-rata): 18.33 detik per pekerja

[Eksekusi] Tugas ke-1 (beban 1s) -> Dialokasikan secara statis ke Pekerja 1
Beban Pekerja 1 sekarang: 1s
[Eksekusi] Tugas ke-2 (beban 2s) -> Dialokasikan secara statis ke Pekerja 2
Beban Pekerja 2 sekarang: 2s
[Eksekusi] Tugas ke-3 (beban 3s) -> Dialokasikan secara statis ke Pekerja 3
Beban Pekerja 3 sekarang: 3s
[Eksekusi] Tugas ke-4 (beban 4s) -> Dialokasikan secara statis ke Pekerja 1
Beban Pekerja 1 sekarang: 5s
[Eksekusi] Tugas ke-5 (beban 5s) -> Dialokasikan secara statis ke Pekerja 2
Beban Pekerja 2 sekarang: 7s
[Eksekusi] Tugas ke-6 (beban 6s) -> Dialokasikan secara statis ke Pekerja 3
Beban Pekerja 3 sekarang: 9s
[Eksekusi] Tugas ke-7 (beban 7s) -> Dialokasikan secara statis ke Pekerja 1
Beban Pekerja 1 sekarang: 12s
[Eksekusi] Tugas ke-8 (beban 8s) -> Dialokasikan secara statis ke Pekerja 2
Beban Pekerja 2 sekarang: 15s
[Eksekusi] Tugas ke-9 (beban 9s) -> Dialokasikan secara statis ke Pekerja 3
Beban Pekerja 3 sekarang: 18s
[Eksekusi] Tugas ke-10 (beban 10s) -> Dialokasikan secara statis ke Pekerja 1
Beban Pekerja 1 sekarang: 22s
>>> [INFO] Pekerja 1 telah mencapai/melewati batas Ideal Distribution! <<<

=== Hasil Akhir Distribusi (Uneven / Tidak Merata) ===
Pekerja 1 total beban: 22s (Selisih dari ideal: 3.67s)
Pekerja 2 total beban: 15s (Selisih dari ideal: 3.33s)
Pekerja 3 total beban: 18s (Selisih dari ideal: 0.33s)
-----
[*] IDEAL DISTRIBUTION POINT: 18.33 detik
-----
PS C:\Users\mling\Documents\Kuliah\kompar\load balancing>

```

Perhitungan Ideal Distribution: Sebelum membagikan tugas, sistem menghitung angka Ideal Distribution dengan membagi total waktu semua tugas dengan jumlah pekerja. Angka ini menjadi patokan apakah distribusi berjalan seimbang atau tidak.

Logika Eksekusi (Static Allocation): Pada setiap baris [Eksekusi], kode menggunakan metode Round-Robin ($\text{index} \% \text{num_workers}$). Artinya, tugas dibagikan secara kaku dan bergiliran (Pekerja 1, lalu 2, lalu 3, lalu kembali ke 1) tanpa peduli apakah tugas tersebut sangat berat atau sangat ringan (uneven).

Mencapai Titik Ideal (Reaching Ideal Distribution): Teks >>> [INFO] Pekerja [X] telah mencapai/melewati batas Ideal Distribution! <<< muncul tepat pada iterasi (when) beban akumulatif seorang pekerja menyentuh atau melebihi angka ideal

yang sudah dihitung di awal. Ini memenuhi syarat soal untuk menunjukkan momen pasti kapan distribusi menyentuh titik tersebut.

Hasil Akhir yang Uneven: Pada bagian hasil akhir, terlihat ada selisih (gap) antara beban pekerja dengan angka Ideal Distribution. Ini membuktikan bahwa pembagian statis pada tugas yang bebannya acak (uneven) akan menghasilkan distribusi akhir yang tidak seimbang (bottleneck pada pekerja tertentu).